

Cell 1 — setup + load the validated tract-year panel for the new cluster-time workflow

In [4]: *# Cell 1 – setup + load the validated tract-year panel for the new cluster-t*

```
from pathlib import Path
import numpy as np
import pandas as pd

pd.set_option("display.max_columns", 200)
pd.set_option("display.width", 200)

# -----
# Paths / core settings
# -----
DATA_DIR = Path("data")
RAW_DIR = DATA_DIR / "raw"
OUT_DIR = DATA_DIR / "processed"

LOW_IGS_THRESHOLD = 45
ANALYSIS_YEARS = list(range(2017, 2025)) # ACS-aligned years only
PRE_COVID_YEARS = [2017, 2018, 2019]
COVID_YEARS = [2020, 2021]
RECOVERY_YEARS = [2022, 2023, 2024]

ECON_VARS = [
    "igs_economy",
    "poverty_rate",
    "unemp_rate",
    "median_household_income",
    "lfpr_16p",
]

CLUSTER_CORE_VARS = [
    "igs_total",
    "igs_economy",
    "poverty_rate",
    "unemp_rate",
    "median_household_income",
    "lfpr_16p",
    "pop_total",
]

DISPLAY_COLS = [
    "display_state",
    "display_county",
    "display_name",
]

REQUIRED_COLS = [
    "geoid",
    "year",
    "igs_total",
```

```

    "igs_economy",
    "poverty_rate",
    "unemp_rate",
    "median_household_income",
    "lfpr_16p",
    "pop_total",
]

PANEL_CANDIDATES = [
    OUT_DIR / "eda_panel_clean.parquet",
    OUT_DIR / "igs_x_acs_full.parquet",
]

def normalize_geoid_series(series: pd.Series) -> pd.Series:
    s = series.astype("string").str.strip()
    numeric = pd.to_numeric(s, errors="coerce")
    numeric_mask = numeric.notna()
    s = s.copy()
    s.loc[numeric_mask] = numeric.loc[numeric_mask].astype("Int64").astype("string")
    s = s.str.replace(r"\.0$", "", regex=True)
    s = s.str.replace(r"\D", "", regex=True)
    s = s.str.zfill(11)
    return s.where(s.str.fullmatch(r"\d{11}"), pd.NA)

def add_display_fields(df: pd.DataFrame) -> pd.DataFrame:
    out = df.copy()

    if "display_state" not in out.columns:
        if "State" in out.columns or "state" in out.columns:
            out["display_state"] = out.get("State", pd.Series(index=out.index, data=out.get("state", pd.Series(index=out.index))))
        else:
            out["display_state"] = pd.NA

    if "display_county" not in out.columns:
        if "County" in out.columns or "county" in out.columns:
            out["display_county"] = out.get("County", pd.Series(index=out.index, data=out.get("county", pd.Series(index=out.index))))
        else:
            out["display_county"] = pd.NA

    if "display_name" not in out.columns:
        if "NAME" in out.columns:
            out["display_name"] = out["NAME"].astype("string")
        elif "Census Tract FIPS code" in out.columns:
            out["display_name"] = "Census Tract " + out["Census Tract FIPS code"].astype("string")
        else:
            out["display_name"] = out["geoid"].astype("string")

    return out

def assign_period(year: int) -> str:
    if year in PRE_COVID_YEARS:
        return "pre_covid"

```

```

if year in COVID_YEARS:
    return "covid"
if year in RECOVERY_YEARS:
    return "recovery"
return "outside_scope"

def load_analysis_panel() -> tuple[pd.DataFrame, str]:
    # Prefer in-memory validated objects if they already exist in the notebook
    if "eda_df" in globals():
        df = eda_df.copy()
        source = "existing global: eda_df"
    elif "model_df_full" in globals():
        df = model_df_full.copy()
        source = "existing global: model_df_full"
    else:
        df = None
        source = None
        for p in PANEL_CANDIDATES:
            if p.exists():
                df = pd.read_parquet(p)
                source = f"file: {p}"
                break
        if df is None:
            searched = ", ".join(str(p) for p in PANEL_CANDIDATES)
            raise FileNotFoundError(
                "Could not find a processed merged panel. "
                f"Expected one of: {searched}"
            )

    # Standardize core keys
    df = df.copy()
    df["geoid"] = normalize_geoid_series(df["geoid"])
    df["year"] = pd.to_numeric(df["year"], errors="coerce").astype("Int64")

    # Restrict to ACS-aligned period only
    df = df[df["year"].isin(ANALYSIS_YEARS)].copy()
    df["year"] = df["year"].astype(int)

    # Add consistent display fields
    df = add_display_fields(df)

    # Remove duplicate tract-year rows if any slipped in
    df = (
        df.sort_values(["geoid", "year"])
        .drop_duplicates(["geoid", "year"])
        .reset_index(drop=True)
    )

    # Validate required columns
    missing = [c for c in REQUIRED_COLS if c not in df.columns]
    if missing:
        raise ValueError(f"Missing required columns for new workflow: {missing}")

    # Add time-period labels for the new method
    df["analysis_period"] = df["year"].map(assign_period)

```

```

    return df, source

panel_df, PANEL_SOURCE = load_analysis_panel()

analysis_cols = (
    ["geoid", "year", "analysis_period"] +
    DISPLAY_COLS +
    CLUSTER_CORE_VARS
)

analysis_panel = panel_df[analysis_cols].copy()

print("Loaded panel from:", PANEL_SOURCE)
print("Analysis panel shape:", analysis_panel.shape)
print("Unique tracts:", analysis_panel["geoid"].nunique())
print("Years:", sorted(analysis_panel["year"].unique().tolist()))
print("Periods:", analysis_panel["analysis_period"].value_counts(dropna=False))
print()
print("Missingness in core vars:")
print(analysis_panel[CLUSTER_CORE_VARS].isna().mean().sort_values(ascending=False))
print()
print(analysis_panel.head())

```

Loaded panel from: file: data\processed\eda_panel_clean.parquet
 Analysis panel shape: (680256, 13)
 Unique tracts: 85032
 Years: [2017, 2018, 2019, 2020, 2021, 2022, 2023, 2024]
 Periods: {'pre_covid': 255096, 'recovery': 255096, 'covid': 170064}

Missingness in core vars:

```
median_household_income    0.116411
poverty_rate                0.111802
unemp_rate                 0.111357
lfpr_l6p                   0.109857
pop_total                  0.104783
igs_total                  0.008863
igs_economy                0.001313
dtype: float64
```

	geoid	year	analysis_period	display_state	display_county	display_name	igs_total	igs_economy	poverty_rate	unemp_rate	median_household_income	lfpr_l6p
0	01001020100	2017	pre_covid	Alabama	Autauga County	Census Tract 201, Autauga County, Alabama	67826.0	0.622387	47.0	34.0	0.106775	0.045504
1	01001020100	2018	pre_covid	Alabama	Autauga County	Census Tract 201, Autauga County, Alabama	58625.0	0.618438	52.0	48.0	0.113365	0.036232
2	01001020100	2019	pre_covid	Alabama	Autauga County	Census Tract 201, Autauga County, Alabama	60208.0	0.606061	46.0	40.0	0.166583	0.028571
3	01001020100	2020	covid	Alabama	Autauga County	Census Tract 201, Autauga County, Alabama	60388.0	0.546855	46.0	48.0	0.136528	0.021127
4	01001020100	2021	covid	Alabama	Autauga County	Census Tract 201, Autauga County, Alabama	57399.0	0.534035	38.0	32.0	0.153546	0.024967

```
pop_total
0    1845.0
1    1923.0
2    1993.0
3    1941.0
4    1791.0
```

Cell 2 — reuse or rebuild tract -> cluster mapping from the validated adjacency work

```
In [5]: # Cell 2 – reuse or rebuild tract -> cluster mapping from the validated adjacency work

from pathlib import Path
from collections import defaultdict, deque
from itertools import combinations
import zipfile
import requests

import geopandas as gpd
```

```

MIN_CLUSTER_TRACTS = 3
MIN_CLUSTER_POP = 5000

TRACT_BASE_URL = "https://www2.census.gov/geo/tiger/TIGER2024/TRACT/"
TRACT_ROOT_DIR = RAW_DIR / "tl_2024_state_tracts"
TRACT_ROOT_DIR.mkdir(parents=True, exist_ok=True)

ADJACENCY_FILE_CANDIDATES = [
    OUT_DIR / "adjacency_gdf.parquet",
    OUT_DIR / "adjacency_scope.parquet",
    OUT_DIR / "cluster_scope_df.parquet",
    OUT_DIR / "tract_cluster_map.parquet",
]

def _load_first_existing_table(paths):
    for p in paths:
        if p.exists():
            try:
                if p.suffix.lower() == ".parquet":
                    return pd.read_parquet(p), str(p)
                elif p.suffix.lower() == ".csv":
                    return pd.read_csv(p), str(p)
            except Exception:
                pass
    return None, None

def build_adjacency_edges(gdf: gpd.GeoDataFrame) -> pd.DataFrame:
    edge_rows = []
    for (state_name, county_name), grp in gdf.groupby(["display_state", "display_county"]):
        grp = grp.reset_index(drop=True).copy()
        if len(grp) < 2:
            continue

        for i, j in combinations(range(len(grp)), 2):
            geom_i = grp.loc[i, "geometry"]
            geom_j = grp.loc[j, "geometry"]

            if geom_i is None or geom_j is None:
                continue
            if geom_i.is_empty or geom_j.is_empty:
                continue

            if geom_i.touches(geom_j):
                edge_rows.append({
                    "display_state": state_name,
                    "display_county": county_name,
                    "geoid_1": grp.loc[i, "geoid"],
                    "geoid_2": grp.loc[j, "geoid"],
                })

    return pd.DataFrame(edge_rows)

def connected_components_from_edges(nodes, edges_df):
    neighbors = defaultdict(set)
    for n in nodes:
        neighbors[n] = set()

```

```

if edges_df is not None and len(edges_df) > 0:
    for _, row in edges_df.iterrows():
        a = str(row["geoid_1"])
        b = str(row["geoid_2"])
        neighbors[a].add(b)
        neighbors[b].add(a)

visited = set()
node_to_cluster_num = {}
cluster_num = 0

for n in nodes:
    if n in visited:
        continue
    cluster_num += 1
    q = deque([n])
    visited.add(n)

    while q:
        cur = q.popleft()
        node_to_cluster_num[cur] = cluster_num
        for nxt in neighbors[cur]:
            if nxt not in visited:
                visited.add(nxt)
                q.append(nxt)

return node_to_cluster_num

def load_state_tract_geometry(state_fips_list):
    tract_gdf_list = []

    for st in state_fips_list:
        zip_name = f"tl_2024_{st}_tract.zip"
        shp_name = f"tl_2024_{st}_tract.shp"
        zip_path = TRACT_ROOT_DIR / zip_name
        extract_dir = TRACT_ROOT_DIR / f"tl_2024_{st}_tract"
        shp_path = extract_dir / shp_name

        if not shp_path.exists():
            extract_dir.mkdir(parents=True, exist_ok=True)

            if not zip_path.exists():
                url = f"{TRACT_BASE_URL}{zip_name}"
                print(f"Downloading {url}")
                r = requests.get(url, timeout=120)
                r.raise_for_status()
                with open(zip_path, "wb") as f:
                    f.write(r.content)

            print(f"Extracting {zip_name}")
            with zipfile.ZipFile(zip_path, "r") as zf:
                zf.extractall(extract_dir)

        if not shp_path.exists():
            raise FileNotFoundError(f"Missing shapefile after extraction: {shp_path}")

```

```

        gdf_state = gpd.read_file(shp_path)
        tract_gdf_list.append(gdf_state)

    tract_gdf = pd.concat(tract_gdf_list, ignore_index=True)
    tract_gdf = gpd.GeoDataFrame(tract_gdf, geometry="geometry", crs=tract_gdf.crs)
    tract_gdf["geoid"] = normalize_geoid_series(tract_gdf["GEOID"])
    tract_gdf = tract_gdf.dropna(subset=["geoid", "geometry"]).copy()
    tract_gdf = tract_gdf[tract_gdf.geometry.notna()].copy()
    tract_gdf = tract_gdf[~tract_gdf.geometry.is_empty].copy()
    tract_gdf = tract_gdf.drop_duplicates(subset=["geoid"]).copy()

    return tract_gdf

def get_cluster_scope_df():
    # Best case: already available from prior notebook work
    if "adjacency_gdf" in globals():
        scope = adjacency_gdf.copy()
        return scope, "existing global: adjacency_gdf"

    if "CLUSTER_SCOPE_DF" in globals():
        scope = CLUSTER_SCOPE_DF.copy()
        return scope, "existing global: CLUSTER_SCOPE_DF"

    disk_df, disk_source = _load_first_existing_table(ADJACENCY_FILE_CANDIDATES)
    if disk_df is not None:
        return disk_df.copy(), f"file: {disk_source}"

    raise RuntimeError(
        "Could not find prior adjacency scope outputs. "
        "Expected one of: adjacency_gdf, CLUSTER_SCOPE_DF, or a saved process"
    )

def build_or_reuse_tract_cluster_map():
    scope_df, scope_source = get_cluster_scope_df()
    scope_df = scope_df.copy()

    if "geoid" not in scope_df.columns:
        raise ValueError("Adjacency scope data is missing 'geoid'.")

    scope_df["geoid"] = normalize_geoid_series(scope_df["geoid"])
    scope_df = add_display_fields(scope_df)
    scope_df = scope_df.dropna(subset=["geoid"]).drop_duplicates(subset=["geoid"])

    # If cluster_id already exists, just standardize and reuse it
    if "cluster_id" in scope_df.columns:
        tract_cluster_map = scope_df[
            ["geoid", "cluster_id", "display_state", "display_county", "display_name"]
        ].drop_duplicates(subset=["geoid"]).copy()

        tract_cluster_map["n_cluster_tracts"] = (
            tract_cluster_map.groupby("cluster_id")["geoid"].transform("nunique")
        )

    return tract_cluster_map, scope_source + " (cluster_id reused)"

```



```

# Otherwise we need geometry to rebuild cluster_id
if "geometry" in scope_df.columns:
    adjacency_scope_gdf = gpd.GeoDataFrame(scope_df, geometry="geometry")
    adjacency_scope_gdf = adjacency_scope_gdf[adjacency_scope_gdf.geometry
    adjacency_scope_gdf = adjacency_scope_gdf[~adjacency_scope_gdf.geometry
else:
    needed_state_fips = sorted(scope_df["geoid"].astype(str).str[:2].dropna())
    tract_gdf = load_state_tract_geometry(needed_state_fips)

    adjacency_scope_gdf = tract_gdf.merge(
        scope_df,
        on="geoid",
        how="inner",
        validate="1:1",
    )
    adjacency_scope_gdf = adjacency_scope_gdf[adjacency_scope_gdf.geometry
    adjacency_scope_gdf = adjacency_scope_gdf[~adjacency_scope_gdf.geometry

adjacency_edges = build_adjacency_edges(adjacency_scope_gdf)
all_nodes = adjacency_scope_gdf["geoid"].astype(str).unique().tolist()
node_to_cluster_num = connected_components_from_edges(all_nodes, adjacency_edges)

adjacency_scope_gdf["cluster_id_num"] = adjacency_scope_gdf["geoid"].astype(str)
adjacency_scope_gdf["cluster_id"] = (
    adjacency_scope_gdf["display_state"].astype(str)
    + " | "
    + adjacency_scope_gdf["display_county"].astype(str)
    + " | cluster_"
    + adjacency_scope_gdf["cluster_id_num"].astype(str)
)

tract_cluster_map = adjacency_scope_gdf[
    ["geoid", "cluster_id", "display_state", "display_county", "display_state"]
].drop_duplicates(subset=["geoid"]).copy()

tract_cluster_map["n_cluster_tracts"] = (
    tract_cluster_map.groupby("cluster_id")["geoid"].transform("nunique")
)

return tract_cluster_map, scope_source + " (cluster_id rebuilt)"

tract_cluster_map, CLUSTER_MAP_SOURCE = build_or_reuse_tract_cluster_map()

print("Cluster map source:", CLUSTER_MAP_SOURCE)
print("tract_cluster_map shape:", tract_cluster_map.shape)
print("Unique tracts mapped:", tract_cluster_map["geoid"].nunique())
print("Unique clusters:", tract_cluster_map["cluster_id"].nunique())
print()
print(
    tract_cluster_map[
        ["cluster_id", "geoid", "display_state", "display_county", "display_state"]
    ].head(15)
)

```

```

-----
RuntimeError                                Traceback (most recent call last)
Cell In[5], line 220
    214     tract_cluster_map["n_cluster_tracts"] = (
    215         tract_cluster_map.groupby("cluster_id")["geoid"].transform
    216         ("nunique")
    217     )
    218     return tract_cluster_map, scope_source + " (cluster_id rebuilt)"
--> 220 tract_cluster_map, CLUSTER_MAP_SOURCE = build_or_reuse_tract_cluster
_map()
    222 print("Cluster map source:", CLUSTER_MAP_SOURCE)
    223 print("tract_cluster_map shape:", tract_cluster_map.shape)

Cell In[5], line 157, in build_or_reuse_tract_cluster_map()
    156 def build_or_reuse_tract_cluster_map():
--> 157     scope_df, scope_source = get_cluster_scope_df()
    158     scope_df = scope_df.copy()
    160     if "geoid" not in scope_df.columns:

Cell In[5], line 151, in get_cluster_scope_df()
    148 if disk_df is not None:
    149     return disk_df.copy(), f"file: {disk_source}"
--> 151 raise RuntimeError(
    152     "Could not find prior adjacency scope outputs. "
    153     "Expected one of: adjacency_gdf, CLUSTER_SCOPE_DF, or a saved pr
ocessed adjacency file."
    154 )

RuntimeError: Could not find prior adjacency scope outputs. Expected one of:
adjacency_gdf, CLUSTER_SCOPE_DF, or a saved processed adjacency file.

```

Cell 3 — join tract panel to clusters and build a cluster-year panel

```

In [ ]: # Cell 3 – join tract panel to clusters and build a cluster-year panel

def safe_weighted_mean(values: pd.Series, weights: pd.Series):
    values = pd.to_numeric(values, errors="coerce")
    weights = pd.to_numeric(weights, errors="coerce")

    mask = values.notna() & weights.notna() & (weights > 0)
    if mask.sum() == 0:
        valid = values.notna()
        if valid.sum() == 0:
            return np.nan
        return values.loc[valid].mean()

    return np.average(values.loc[mask], weights=weights.loc[mask])

clustered_panel = (
    analysis_panel
    .merge(
        tract_cluster_map[
            ["geoid", "cluster_id", "display_state", "display_county", "dis
"],

```

```

        on="geoid",
        how="inner",
        validate="many_to_one",
        suffixes=("", "_cluster")
    )
    .copy()
)

# Prefer cluster metadata from the tract_cluster_map
clustered_panel["cluster_display_state"] = clustered_panel["display_state_cl
clustered_panel["cluster_display_county"] = clustered_panel["display_county_

cluster_year_rows = []

for (cluster_id, year, period), grp in clustered_panel.groupby(
    ["cluster_id", "year", "analysis_period"],
    dropna=False
):
    row = {
        "cluster_id": cluster_id,
        "year": int(year),
        "analysis_period": period,
        "display_state": grp["cluster_display_state"].dropna().iloc[0] if gr
        "display_county": grp["cluster_display_county"].dropna().iloc[0] if
        "n_cluster_tracts": grp["geoid"].nunique(),
        "cluster_pop_total": pd.to_numeric(grp["pop_total"], errors="coerce"
        "low_igs_tract_share": pd.to_numeric(grp["igs_total"], errors="coerc
    }

    weights = grp["pop_total"]

    for col in CLUSTER_CORE_VARS:
        row[col] = safe_weighted_mean(grp[col], weights)

    cluster_year_rows.append(row)

cluster_year_panel = pd.DataFrame(cluster_year_rows).sort_values(
    ["cluster_id", "year"]
).reset_index(drop=True)

print("clustered_panel shape:", clustered_panel.shape)
print("cluster_year_panel shape:", cluster_year_panel.shape)
print("Unique clusters in panel:", cluster_year_panel["cluster_id"].nunique()
print("Years in cluster panel:", sorted(cluster_year_panel["year"].unique()).
print()
print(cluster_year_panel.head(15))

```

Cell 4 — build period-level summaries and transparent trajectory features

```

In [ ]: # Cell 4 – build period-level summaries and transparent trajectory features

PERIOD_ORDER = ["pre_covid", "covid", "recovery"]

cluster_period_panel = (

```

```

cluster_year_panel
.groupby(["cluster_id", "analysis_period"], as_index=False)
.agg({
    "display_state": "first",
    "display_county": "first",
    "n_cluster_tracts": "max",
    "cluster_pop_total": "mean",
    "low_igs_tract_share": "mean",
    "igs_total": "mean",
    "igs_economy": "mean",
    "poverty_rate": "mean",
    "unemp_rate": "mean",
    "median_household_income": "mean",
    "lfpr_16p": "mean",
})
)

cluster_period_panel["analysis_period"] = pd.Categorical(
    cluster_period_panel["analysis_period"],
    categories=PERIOD_ORDER,
    ordered=True
)

wide_index_cols = ["cluster_id"]
wide_value_cols = [
    "display_state",
    "display_county",
    "n_cluster_tracts",
    "cluster_pop_total",
    "low_igs_tract_share",
    "igs_total",
    "igs_economy",
    "poverty_rate",
    "unemp_rate",
    "median_household_income",
    "lfpr_16p",
]

cluster_period_wide = (
    cluster_period_panel
    .pivot(index="cluster_id", columns="analysis_period", values=wide_value_
)

cluster_period_wide.columns = [
    f"{metric}_{period}" for metric, period in cluster_period_wide.columns
]
cluster_period_wide = cluster_period_wide.reset_index()

# Pull one stable version of state/county + latest-year population
latest_cluster_meta = (
    cluster_year_panel
    .sort_values(["cluster_id", "year"])
    .groupby("cluster_id", as_index=False)
    .tail(1)[["cluster_id", "display_state", "display_county", "n_cluster_tr
    .rename(columns={"cluster_pop_total": "latest_cluster_pop_total"})
)

```

```

cluster_period_summary = cluster_period_wide.merge(
    latest_cluster_meta,
    on="cluster_id",
    how="left",
    validate="1:1"
)

# ---- trajectory deltas ----
# Lower is worse: igs_total, igs_economy
cluster_period_summary["igs_total_drop_pre_to_covid"] = (
    cluster_period_summary["igs_total_pre_covid"] - cluster_period_summary["igs_total_covid"]
)
cluster_period_summary["igs_economy_drop_pre_to_covid"] = (
    cluster_period_summary["igs_economy_pre_covid"] - cluster_period_summary["igs_economy_covid"]
)
cluster_period_summary["igs_total_rebound_covid_to_recovery"] = (
    cluster_period_summary["igs_total_recovery"] - cluster_period_summary["igs_total_covid"]
)
cluster_period_summary["igs_economy_rebound_covid_to_recovery"] = (
    cluster_period_summary["igs_economy_recovery"] - cluster_period_summary["igs_economy_covid"]
)

# Higher is worse: poverty_rate, unemp_rate
cluster_period_summary["poverty_increase_pre_to_covid"] = (
    cluster_period_summary["poverty_rate_covid"] - cluster_period_summary["poverty_rate_pre_covid"]
)
cluster_period_summary["unemp_increase_pre_to_covid"] = (
    cluster_period_summary["unemp_rate_covid"] - cluster_period_summary["unemp_rate_pre_covid"]
)
cluster_period_summary["poverty_improve_covid_to_recovery"] = (
    cluster_period_summary["poverty_rate_covid"] - cluster_period_summary["poverty_rate_pre_covid"]
)
cluster_period_summary["unemp_improve_covid_to_recovery"] = (
    cluster_period_summary["unemp_rate_covid"] - cluster_period_summary["unemp_rate_pre_covid"]
)

# Higher is better: income, labor-force participation
cluster_period_summary["income_drop_pre_to_covid"] = (
    cluster_period_summary["median_household_income_pre_covid"] - cluster_period_summary["median_household_income_covid"]
)
cluster_period_summary["lfpr_drop_pre_to_covid"] = (
    cluster_period_summary["lfpr_16p_pre_covid"] - cluster_period_summary["lfpr_16p_covid"]
)
cluster_period_summary["income_rebound_covid_to_recovery"] = (
    cluster_period_summary["median_household_income_recovery"] - cluster_period_summary["median_household_income_covid"]
)
cluster_period_summary["lfpr_rebound_covid_to_recovery"] = (
    cluster_period_summary["lfpr_16p_recovery"] - cluster_period_summary["lfpr_16p_covid"]
)

# ---- structural filters ----
cluster_period_summary["serious_cluster_flag"] = (
    (cluster_period_summary["n_cluster_tracts"] >= MIN_CLUSTER_TRACTS) &
    (cluster_period_summary["latest_cluster_pop_total"] >= MIN_CLUSTER_POP)
)

```

```

cluster_period_summary["low_igs_recovery_flag"] = (
    cluster_period_summary["igs_total_recovery"] < LOW_IGS_THRESHOLD
)

cluster_period_summary["low_igs_economy_recovery_flag"] = (
    cluster_period_summary["igs_economy_recovery"] < LOW_IGS_THRESHOLD
)

print("cluster_period_panel shape:", cluster_period_panel.shape)
print("cluster_period_summary shape:", cluster_period_summary.shape)
print()
print("Serious cluster count:", int(cluster_period_summary["serious_cluster_
print("Recovery low-IGS cluster count:", int(cluster_period_summary["low_igs
print()
print(
    cluster_period_summary[
        [
            "cluster_id",
            "display_state",
            "display_county",
            "n_cluster_tracts",
            "latest_cluster_pop_total",
            "igs_total_pre_covid",
            "igs_total_covid",
            "igs_total_recovery",
            "poverty_rate_pre_covid",
            "poverty_rate_covid",
            "poverty_rate_recovery",
            "serious_cluster_flag",
            "low_igs_recovery_flag",
        ]
    ].head(15)
)

```

Cell 5 — practical shortlist logic for the new method

In []: *# Cell 5 – practical shortlist logic for the new method*

```

def bad_when_high(series: pd.Series) -> pd.Series:
    s = pd.to_numeric(series, errors="coerce")
    return s.rank(pct=True, method="average")

def bad_when_low(series: pd.Series) -> pd.Series:
    s = pd.to_numeric(series, errors="coerce")
    return (-s).rank(pct=True, method="average")

cluster_scored = cluster_period_summary.copy()

# 1) Still low on IGS in recovery
cluster_scored["score_low_igs_recovery"] = (
    bad_when_low(cluster_scored["igs_total_recovery"]) +
    bad_when_low(cluster_scored["igs_economy_recovery"])
) / 2

```

```

# 2) Economically vulnerable in recovery
cluster_scored["score_econ_vulnerability_recovery"] = (
    bad_when_low(cluster_scored["igs_economy_recovery"]) +
    bad_when_high(cluster_scored["poverty_rate_recovery"]) +
    bad_when_high(cluster_scored["unemp_rate_recovery"]) +
    bad_when_low(cluster_scored["median_household_income_recovery"]) +
    bad_when_low(cluster_scored["lfpr_16p_recovery"])
) / 5

# 3) Harder COVID shock relative to pre-COVID baseline
cluster_scored["score_covid_shock"] = (
    bad_when_high(cluster_scored["igs_total_drop_pre_to_covid"]) +
    bad_when_high(cluster_scored["igs_economy_drop_pre_to_covid"]) +
    bad_when_high(cluster_scored["poverty_increase_pre_to_covid"]) +
    bad_when_high(cluster_scored["unemp_increase_pre_to_covid"]) +
    bad_when_high(cluster_scored["income_drop_pre_to_covid"]) +
    bad_when_high(cluster_scored["lfpr_drop_pre_to_covid"])
) / 6

# 4) Weak recovery since COVID
cluster_scored["score_weak_recovery"] = (
    bad_when_low(cluster_scored["igs_total_rebound_covid_to_recovery"]) +
    bad_when_low(cluster_scored["igs_economy_rebound_covid_to_recovery"]) +
    bad_when_low(cluster_scored["poverty_improve_covid_to_recovery"]) +
    bad_when_low(cluster_scored["unemp_improve_covid_to_recovery"]) +
    bad_when_low(cluster_scored["income_rebound_covid_to_recovery"]) +
    bad_when_low(cluster_scored["lfpr_rebound_covid_to_recovery"])
) / 6

# Transparent final score
cluster_scored["cluster_shortlist_score"] = (
    0.35 * cluster_scored["score_low_igs_recovery"] +
    0.35 * cluster_scored["score_econ_vulnerability_recovery"] +
    0.15 * cluster_scored["score_covid_shock"] +
    0.15 * cluster_scored["score_weak_recovery"]
)

# Final practical screen for enrichment candidates
cluster_shortlist = (
    cluster_scored
    .loc[
        cluster_scored["serious_cluster_flag"] &
        cluster_scored["low_igs_recovery_flag"]
    ]
    .sort_values(
        [
            "cluster_shortlist_score",
            "igs_total_recovery",
            "igs_economy_recovery",
            "latest_cluster_pop_total",
        ],
        ascending=[False, True, True, False]
    )
    .reset_index(drop=True)
)

```

```

cluster_shortlist["rank"] = np.arange(1, len(cluster_shortlist) + 1)

shortlist_view_cols = [
    "rank",
    "cluster_id",
    "display_state",
    "display_county",
    "n_cluster_tracts",
    "latest_cluster_pop_total",
    "igs_total_pre_covid",
    "igs_total_covid",
    "igs_total_recovery",
    "igs_economy_recovery",
    "poverty_rate_recovery",
    "unemp_rate_recovery",
    "median_household_income_recovery",
    "lfpr_l6p_recovery",
    "score_low_igs_recovery",
    "score_econ_vulnerability_recovery",
    "score_covid_shock",
    "score_weak_recovery",
    "cluster_shortlist_score",
]

print("Final shortlist shape:", cluster_shortlist.shape)
print()
print(cluster_shortlist[shortlist_view_cols].head(20))

# Optional: persist key outputs for later healthcare + business enrichment
OUT_DIR.mkdir(parents=True, exist_ok=True)
cluster_year_panel.to_parquet(OUT_DIR / "cluster_year_panel.parquet", index=False)
cluster_period_summary.to_parquet(OUT_DIR / "cluster_period_summary.parquet", index=False)
cluster_shortlist.to_parquet(OUT_DIR / "cluster_shortlist.parquet", index=False)

print()
print("Saved:")
print("-", OUT_DIR / "cluster_year_panel.parquet")
print("-", OUT_DIR / "cluster_period_summary.parquet")
print("-", OUT_DIR / "cluster_shortlist.parquet")

```

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []: